

Taller de Patrones: Microkernel, Tuberías y Filtros



Universidad
del Cauca

Vigilada Mineducación

Laboratorio de Ingeniería de Software II

Banco de Preguntas Saber PRO - Grupo [X]

Presentado por:

Miguel Angel Meza Lopez
Oscar Alejandro Menza Chaviznan

Profesor:

Pablo Magé
Paola Bedoya

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Ingeniería de Sistemas

Popayán, septiembre de 2026

CONTENIDO

Introducción.....	3
Desarrollo.....	4
1. Descripción del problema.....	4
2. Patrón Microkernel	4
2.1. Plugins implementados.....	4
3. Patrón Tuberías y Filtros	4
3.1. Filtros del pipeline	4
4. Estructura del proyecto	5
5. Tecnologías utilizadas	5
6. Pruebas unitarias.....	5
7. Repositorio GitHub	6
8. Conclusiones.....	6
Referencias	7

Introducción

El presente documento describe el desarrollo del taller de patrones de arquitectura de software del curso Laboratorio de Ingeniería de Software II. El objetivo es implementar una solución extensible para el Banco de Preguntas Saber PRO aplicando el patrón Microkernel (Plug-in) en combinación con el patrón Tuberías y Filtros (Pipes and Filters). La aplicación se desarrolló en Java con una interfaz de escritorio en Swing e incluye pruebas unitarias automatizadas con JUnit.

Desarrollo

1. Descripción del problema

El Banco de Preguntas Saber PRO trabajaba inicialmente solo con preguntas de selección múltiple. La institución requiere incorporar progresivamente nuevos tipos de preguntas (análisis de casos, escenarios, recursos multimedia, generadas automáticamente e importadas desde sistemas externos) sin modificar el núcleo del sistema cada vez que aparece un nuevo tipo. Esta necesidad de extensibilidad y bajo acoplamiento justifica el uso del patrón Microkernel.

2. Patrón Microkernel

El núcleo del sistema, implementado en la clase `QuestionMicrokernel`, almacena el banco de preguntas en una estructura `Map<String, Question>`, registra los plugins y los ejecuta. Los plugins se cargan dinámicamente mediante Reflexión, leyendo los nombres de las clases desde el archivo de configuración `plugins.properties`; de esta forma el núcleo no depende de las clases concretas de los plugins. Este mecanismo se basó en el ejemplo visto en clase teórica sobre el envío de paquetes a distintos países. Todos los plugins cumplen el contrato de la interfaz `QuestionPlugin`, que define los métodos `getName`, `supports` y `generate`.

2.1. Plugins implementados

- `MultipleChoiceQuestionPlugin`: genera preguntas de selección múltiple e integra el pipeline de validación.
- `CaseQuestionPlugin`: genera preguntas basadas en análisis de casos.
- `MultimediaQuestionPlugin`: genera preguntas que incluyen recursos multimedia (imagen, audio o video).

Para agregar un nuevo tipo de pregunta basta con crear una nueva clase que implemente `QuestionPlugin` y registrarla en `plugins.properties`, sin modificar el núcleo.

3. Patrón Tuberías y Filtros

Para validar las preguntas antes de almacenarlas se implementó el patrón Tuberías y Filtros. Un pipeline (clase `QuestionPipeline`) ejecuta de forma secuencial una serie de filtros; si alguno rechaza la solicitud, la tubería se detiene y la pregunta no se genera. Este pipeline se integró dentro del plugin `MultipleChoiceQuestionPlugin`.

3.1. Filtros del pipeline

- `ContentValidationFilter`: valida que el título y el contenido no estén vacíos y cumplan una longitud mínima.

- OptionsValidationFilter: valida que existan exactamente cuatro opciones, sin opciones vacías ni duplicadas.
- ClassificationFilter: valida que la clasificación (competencia o área de conocimiento) sea válida.
- CorrectAnswerValidationFilter: valida que exista una respuesta correcta y que pertenezca a la lista de opciones.

4. Estructura del proyecto

El proyecto se organizó en paquetes según su responsabilidad, siguiendo el principio de responsabilidad única (SRP):

microkernel.app -> Main y MainFrame (interfaz Swing).

microkernel.entities -> Question y QuestionRequest (modelo de dominio).

microkernel.interfaces -> QuestionPlugin (contrato de los plugins).

microkernel.core -> QuestionMicrokernel (núcleo y carga por reflexión).

microkernel.pipeline -> QuestionFilter y QuestionPipeline.

microkernel.filters -> los cuatro filtros de validación.

microkernel.plugins -> MultipleChoice, Case y Multimedia.

resources -> plugins.properties. test -> pruebas unitarias JUnit.

5. Tecnologías utilizadas

- Lenguaje: Java (JDK 17 o superior).
- Interfaz de escritorio: Java Swing.
- Gestión de dependencias: Maven.
- Pruebas unitarias: JUnit 5.
- Carga dinámica de plugins: Reflexión de Java (Class.forName).

6. Pruebas unitarias

Se implementaron pruebas unitarias automatizadas con JUnit 5 para validar el comportamiento de las clases del dominio. Se probó cada uno de los cuatro filtros con casos válidos e inválidos, el pipeline completo (aceptando una solicitud válida y rechazando una inválida) y el núcleo del microkernel: la carga de los tres plugins por reflexión, la generación y almacenamiento de preguntas válidas, el rechazo de preguntas que no pasan el pipeline y el lanzamiento de una excepción cuando no existe un plugin para el tipo solicitado.

7. Repositorio GitHub

El código fuente completo y las pruebas unitarias se encuentran disponibles en el siguiente repositorio público de GitHub:

https://github.com/MiguelMeza17/banco-preguntas_Microkernel

8. Conclusiones

La combinación de los patrones Microkernel y Tuberías y Filtros permitió construir una solución extensible y de bajo acoplamiento. Nuevos tipos de preguntas pueden agregarse como plugins sin modificar el núcleo, y las reglas de validación pueden componerse y reutilizarse como filtros independientes. El uso de la reflexión mostró cómo un sistema puede descubrir y cargar componentes en tiempo de ejecución a partir de configuración externa, favoreciendo el principio abierto/cerrado.

Referencias

Pantoja, W. L. y Hurtado, J. A. Material de apoyo del curso Laboratorio de Ingeniería de Software II. Universidad del Cauca, 2026.